

Multiple brokers

Docker command

1. To start kafka process

- Start yml process
 - `docker-compose up -d`

2. To create a new topic

- Start the broker (kafka1)
 - `docker exec -it kafka1 /bin/bash`
- Create a new topic
 - `kafka-topics --bootstrap-server localhost:9092 --create --topic randomTopic --partitions 2 --replication-factor 3`

3. To create a producer

- Start the broker (kafka1)
 - `docker exec -it kafka1 /bin/bash`
- Start producer (also use kafka1 broker)
 - `kafka-console-producer --bootstrap-server localhost:9092 --topic randomTopic`
> Type somethings

4. To create a consumer

- Open a new terminal
- Start the broker (kafka2)
 - `docker exec -it kafka2 /bin/bash`
- Start consuming
 - `kafka-console-consumer --bootstrap-server localhost:9092 --topic randomTopic --from-beginning`

5. Use Python as a producer and consumer

- `pip install confluent_kafka`
- Run “python Consumer.py”
- Run “python Producer.py”

Question

1. What happens if one broker is shut down?
2. What are the maximum partitions you can set?
3. What are the maximum replication factors you can set?

Exercise

1. Experiment to send key/value events?
 - a. `kafka-console-producer --bootstrap-server localhost:9092 --topic t1 --property "parse.key=true" --property "key.separator=" > c001,700`
 - b. How to modify the `kafka-console-consumer` to receive the event including key? Hint: type `kafka-console-consumer -h` to see all possible parameters.
2. Create a new topic 't1' with 1024 partitions, and 5 replication factors. Hint: you need to modify `docker-compose.yml` and restart it again.

Miscellaneous Docker commands

List all topics

```
kafka-topics --bootstrap-server localhost:9092 --list
```

Describe a topic

```
kafka-topics --bootstrap-server localhost:9092 --describe --topic randomTopic
```

Delete a topic

```
kafka-topics --bootstrap-server localhost:9092 --delete --topic <topic_name>
```

Reference

3. <https://docs.ksqldb.io/en/latest/concepts/apache-kafka-primer/>
4. More understanding about partition and offset
<https://developer.confluent.io/tutorials/kafka-console-consumer-read-specific-offsets-partitions/kafka.html>